Experiment 11 — K-Means Text Clustering

Objective

To group similar text documents into clusters based on TF-IDF feature similarity using **KMeans from Scikit-learn**.

Python Code

```
# Experiment 11: K-Means Text Clustering

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
import pandas as pd

# Sample documents
documents = [
    "smartphone camera processor battery mobile phone",
    "mobile phone smartphone camera internet connectivity",
    "smartphone processor battery camera technology",

    "football team player match goal stadium",
    "football player scored goals team championship",
    "football training player team fitness goals",

    "online shopping customer product price discount",
    "customers compare product prices shopping online",
    "ecommerce shopping products discount delivery customer"
]

# Convert documents to TF-IDF vectors
vectorizer = TfidfVectorizer(stop_words="english")
X = vectorizer.fit_transform(documents)

# Apply K-Means
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(X)

# Display results
print("Number of documents:", X.shape[0])
print("Number of TF-IDF features:", X.shape[1])

for i, label in enumerate(labels, 1):
    print(f"Document {i}: Cluster {label}")

# Display important terms
```

```
terms = vectorizer.get_feature_names_out()
order_centroids = kmeans.cluster_centers_.argsort()[:, ::-1]

print("\nTop terms in each cluster:")

for cluster in range(3):
    top_terms = [
        terms[index]
        for index in order_centroids[cluster, :5]
    ]

    print(
        f"Cluster {cluster}: "
        + ", ".join(top_terms)
    )

print("\nCluster sizes:")
print(pd.Series(labels).value_counts().sort_index())
```

Output

```
Number of documents: 9
Number of TF-IDF features: 32

Document 1: Cluster 2
Document 2: Cluster 2
Document 3: Cluster 2

Document 4: Cluster 0
Document 5: Cluster 0
Document 6: Cluster 0

Document 7: Cluster 1
Document 8: Cluster 1
Document 9: Cluster 1

Top terms in each cluster:
Cluster 0: team, player, football, goals, championship
Cluster 1: shopping, product, online, discount, customer
Cluster 2: smartphone, camera, battery, processor, phone

Cluster sizes:
0    3
1    3
2    3
```

Interpretation

Cluster	Discovered Theme	Documents
Cluster 0	Football/Sports	3

Cluster	Discovered Theme	Documents
Cluster 1	Online Shopping/E-commerce	3
Cluster 2	Smartphones/Technology	3

Result: K-Means successfully grouped the documents into three meaningful clusters based on textual similarity.

Experiment 12 — LDA Topic Modeling Using Gensim

Objective

To discover hidden topics from a collection of documents using **Latent Dirichlet Allocation (LDA)**.

Python Code

```
# Experiment 12: LDA Topic Modeling using Gensim

from gensim import corpora
from gensim.models import LdaModel

documents = [
    "football team player match goal training stadium",
    "football player scored goal team championship match",
    "football training player team goal stadium",

    "machine learning artificial intelligence model data
algorithm",
    "deep learning neural network artificial intelligence
model",
    "machine learning data science model prediction
algorithm",

    "online shopping customer product price discount
delivery",
    "ecommerce shopping product online payment",
    "online shopping customers products discounts delivery"
]

# Tokenization
texts = [
    document.lower().split()
    for document in documents
]

# Create dictionary
dictionary = corpora.Dictionary(texts)
```

```
# Convert documents into Bag-of-Words
corpus = [
    dictionary.doc2bow(text)
    for text in texts
]

# Create LDA model
lda_model = LdaModel(
    corpus=corpus,
    id2word=dictionary,
    num_topics=3,
    random_state=42,
    passes=20,
    iterations=100
)

print("Number of documents:", len(texts))
print("Vocabulary size:", len(dictionary))

# Display discovered topics
print("\nDiscovered Topics:")

for topic_id, topic_words in lda_model.show_topics(
        num_topics=3,
        num_words=6,
        formatted=False):

    words = [
        word
        for word, probability in topic_words
    ]

    print(
        f"Topic {topic_id}: "
        + ", ".join(words)
    )

# Display dominant topic for every document
print("\nDominant topic for each document:")

for i, bow in enumerate(corpus, 1):

    distribution = lda_model.get_document_topics(
        bow,
        minimum_probability=0
    )

    dominant_topic = max(
        distribution,
        key=lambda x: x[1]
    )
```

```
    print(
        f"Document {i}: "
        f"Topic {dominant_topic[0]} "
        f"(probability={dominant_topic[1]:.3f})"
    )
```

Output

```
Number of documents: 9
Vocabulary size: 33

Topic 0: football, team, goal, player, stadium, training
Topic 1: learning, model, algorithm, intelligence, machine,
data
Topic 2: shopping, online, delivery, product, discount,
customer

Dominant topic for each document:

Document 1: Topic 0 (probability=0.916)
Document 2: Topic 0 (probability=0.916)
Document 3: Topic 0 (probability=0.905)

Document 4: Topic 1 (probability=0.916)
Document 5: Topic 1 (probability=0.916)
Document 6: Topic 1 (probability=0.916)

Document 7: Topic 2 (probability=0.916)
Document 8: Topic 2 (probability=0.888)
Document 9: Topic 2 (probability=0.904)
```

Interpretation

Topic	Main Theme
Topic 0	Football and Sports
Topic 1	Machine Learning and AI
Topic 2	Online Shopping and E-commerce

Result: LDA successfully identified the hidden thematic structures in the document collection.

Experiment 13 — Model Evaluation Metrics

Objective

To evaluate a text classification model using:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- ROC-AUC

Python Code

```
# Experiment 13: Text Classification Evaluation

import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    roc_auc_score,
    classification_report
)

# Text dataset
reviews = [
    "excellent product great quality",
    "amazing service fast delivery",
    "happy purchase great quality",
    "product works perfectly excellent",
    "good value useful product",
    "fantastic quality wonderful service",
    "very good product and service",
    "excellent purchase fast delivery",

    "terrible product bad quality",
    "poor service slow delivery",
    "unhappy purchase poor quality",
    "product failed disappointing",
    "bad value useless product",
    "awful quality horrible service",
    "terrible service bad product",
    "poor quality disappointing purchase"
]

# 1 = Positive
# 0 = Negative
```

```
y = np.array(
    [1] * 8 + [0] * 8
)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    reviews,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# Create pipeline
model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer()
    ),
    (
        "classifier",
        LogisticRegression(
            random_state=42
        )
    )
])

# Train
model.fit(
    X_train,
    y_train
)

# Predictions
y_pred = model.predict(X_test)

# Prediction probabilities
y_probability = model.predict_proba(
    X_test
)[:, 1]

# Evaluation metrics
accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
```

```
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

roc_auc = roc_auc_score(
    y_test,
    y_probability
)

print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))
print("ROC-AUC  :", round(roc_auc, 4))

print("\nConfusion Matrix:")
print(
    confusion_matrix(
        y_test,
        y_pred
    )
)

print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=[
            "Negative",
            "Positive"
        ],
        zero_division=0
    )
)
```

Output

```
Accuracy : 0.6000
Precision: 0.5000
```

```
Recall   : 1.0000
F1-Score : 0.6667
ROC-AUC  : 1.0000

Confusion Matrix:
[[1 2]
 [0 2]]

              precision    recall  f1-score   support

    Negative       1.00      0.33      0.50         3
    Positive       0.50      1.00      0.67         2

    accuracy                           0.60         5
   macro avg       0.75      0.67      0.58         5
weighted avg       0.80      0.60      0.57         5
```

Interpretation

The confusion matrix is:

```
                Predicted
                Neg   Pos

Actual Neg       1     2
Actual Pos       0     2
```

This small demonstration dataset produces different metric values, illustrating why **Accuracy alone should not be used** to evaluate a classification model.

Experiment 14 — Explainable AI Using SHAP

Objective

To use **SHAP (SHapley Additive exPlanations)** to understand how individual words/features contribute to a machine-learning prediction.

Python Code

```
# Experiment 14: Explainable AI using SHAP

import numpy as np
import shap

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

texts = [
```

```
    "excellent amazing wonderful great product",
    "good useful quality fast service",
    "fantastic excellent product performance",
    "happy with purchase",

    "bad terrible awful product",
    "poor quality disappointing service",
    "horrible useless product experience",
    "unhappy purchase",

    "excellent quality good service",
    "terrible quality bad service"
]

labels = np.array([
    1, 1, 1, 1,
    0, 0, 0, 0,
    1, 0
])

# TF-IDF
vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(
    texts
)

# Train classifier
model = LogisticRegression(
    random_state=42
)

model.fit(
    X,
    labels
)

# SHAP explainer
explainer = shap.LinearExplainer(
    model,
    X
)

shap_values = explainer(X)

# Explain first document
document_index = 0

feature_names = (
    vectorizer
    .get_feature_names_out()
```

```
)

values = shap_values.values[
    document_index
]

contributions = sorted(
    zip(feature_names, values),
    key=lambda x: abs(x[1]),
    reverse=True
)

print("Text explained:")
print(texts[document_index])

print("\nTop SHAP feature contributions:")

for word, value in contributions[:8]:

    print(
        f"{word:15s} "
        f"{value:+.4f}"
    )

prediction = model.predict(
    X[document_index]
)[0]

probability = model.predict_proba(
    X[document_index]
)[0, 1]

print(
    "\nPrediction:",
    "Positive" if prediction == 1
    else "Negative"
)

print(
    "Positive probability:",
    round(probability, 4)
)
```

Output

```
Text explained:
excellent amazing wonderful great product

Top SHAP feature contributions:
excellent       +0.1224
amazing         +0.0874
```

```
great           +0.0874
wonderful       +0.0874
bad             +0.0421
terrible        +0.0421
good            -0.0418
unhappy         +0.0247

Prediction: Positive
Positive probability: 0.6124
```

Interpretation

SHAP helps answer:

"Why did the model make this prediction?"

For the demonstrated document, words such as **excellent, amazing, great,** and **wonderful** contribute positively toward the model's prediction.

This is particularly useful for:

- Sentiment analysis
- Spam detection
- Fake-news classification
- Medical text classification
- Customer-review analysis
- Explainable NLP systems

Important: SHAP values explain the behavior of the particular trained model and dataset; they should not automatically be interpreted as causal effects.

Experiment 15 — End-to-End Text Classification Pipeline

Objective

To develop a complete NLP pipeline consisting of:

```
Raw Text
    ↓
NLTK Preprocessing
    ↓
TF-IDF Feature Extraction
    ↓
Logistic Regression
    ↓
Prediction
    ↓
Flask API
```

Python Code

```
# Experiment 15
# End-to-End Text Classification using NLTK,
# Scikit-learn and Flask

import re
import nltk
import numpy as np

from nltk.corpus import stopwords

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

from flask import Flask, request, jsonify


# ------------------------------------------------
# STEP 1: Download NLTK Stopwords
# ------------------------------------------------

nltk.download(
    "stopwords",
    quiet=True
)

stop_words = set(
    stopwords.words("english")
)


# ------------------------------------------------
# STEP 2: Text Preprocessing
# ------------------------------------------------

def preprocess_text(text):

    # Convert to lowercase
    text = text.lower()

    # Remove punctuation and numbers
    text = re.sub(
        r"[^a-z\s]",
        " ",
        text
    )

    # Tokenization
    tokens = text.split()
```

```
    # Remove stop words
    tokens = [
        word
        for word in tokens
        if word not in stop_words
    ]

    return " ".join(tokens)


# ------------------------------------------------
# STEP 3: Dataset
# ------------------------------------------------

texts = [
    "excellent product good quality",
    "amazing service fast delivery",
    "great useful product",
    "happy purchase",
    "fantastic quality excellent service",

    "terrible product bad quality",
    "poor service slow delivery",
    "unhappy purchase",
    "disappointing useless product",
    "awful quality horrible service"
]

labels = np.array([
    1, 1, 1, 1, 1,
    0, 0, 0, 0, 0
])


# ------------------------------------------------
# STEP 4: Preprocess Dataset
# ------------------------------------------------

cleaned_texts = [
    preprocess_text(text)
    for text in texts
]


# ------------------------------------------------
# STEP 5: Build ML Pipeline
# ------------------------------------------------

model = Pipeline([
    (
```

```
        "tfidf",
        TfidfVectorizer()
    ),

    (
        "classifier",
        LogisticRegression(
            random_state=42
        )
    )
])


# ------------------------------------------------
# STEP 6: Train Model
# ------------------------------------------------

model.fit(
    cleaned_texts,
    labels
)


# ------------------------------------------------
# STEP 7: Test Model
# ------------------------------------------------

test_texts = [
    "excellent product with great quality",
    "terrible service and poor product"
]

for text in test_texts:

    cleaned = preprocess_text(
        text
    )

    prediction = model.predict(
        [cleaned]
    )[0]

    probability = model.predict_proba(
        [cleaned]
    )[0, 1]

    print("Original:", text)
    print("Cleaned :", cleaned)

    print(
        "Prediction:",
```

```
        "Positive"
        if prediction == 1
        else "Negative"
    )

    print(
        "Positive probability:",
        round(probability, 3)
    )

    print()


# ------------------------------------------------
# STEP 8: Flask Application
# ------------------------------------------------

app = Flask(__name__)


@app.route(
    "/predict",
    methods=["POST"]
)
def predict():

    data = request.get_json()

    text = data.get(
        "text",
        ""
    )

    cleaned = preprocess_text(
        text
    )

    prediction = model.predict(
        [cleaned]
    )[0]

    probability = model.predict_proba(
        [cleaned]
    )[0, 1]

    return jsonify({

        "text": text,

        "prediction":
            "Positive"
```

```
            if prediction == 1
            else "Negative",

        "positive_probability":
            round(
                float(probability),
                4
            )
    })


# ------------------------------------------------
# STEP 9: Run Flask
# ------------------------------------------------

if __name__ == "__main__":

    app.run(
        host="0.0.0.0",
        port=5000,
        debug=True
    )
```

Executed ML Pipeline Output

```
Original: excellent product with great quality
Cleaned : excellent product great quality
Prediction: Positive
Positive probability: 0.593

Original: terrible service and poor product
Cleaned : terrible service poor product
Prediction: Negative
Positive probability: 0.424
```

The preprocessing, TF-IDF feature extraction, Logistic Regression training, and prediction stages were executed successfully.

Flask API

Once Flask is installed on your computer, run:

```
pip install flask nltk scikit-learn numpy pandas
```

Then:

```
python experiment15.py
```

The API will run at:

```
http://127.0.0.1:5000
```

You can send a POST request such as:

```
{
    "text": "excellent product and fast service"
}
```

and receive a response such as:

```
{
    "text": "excellent product and fast service",
    "prediction": "Positive",
    "positive_probability": 0.XXXX
}
```